

Smart Property AI

Smart Property AI is a multilingual and accessible property-management and apartment-search platform built with FastAPI, SQLAlchemy, SQLite, Jinja2, HTML, CSS, and JavaScript.

It supports public apartment discovery, landlord property management, tenant services, rental applications, appointments, maintenance requests, complaints, payments, documents, notifications, and conversational assistance.

Features

Multilingual Conversational AI

- Multilingual assistance
- Voice-enabled forms
- Browser speech output
- Accessible keyboard and screen-reader support
- Apartment-search guidance
- Tenant and landlord assistance
- AI and privacy disclosures

The current assistant uses local rule-based responses. A trained model can later replace the local response engine without changing the frontend routes.

Listings and Guided Search

- Public apartment listings
- Search by city and property type
- Minimum and maximum monthly rent
- Bedroom filtering
- Wheelchair-accessibility filtering
- Pet-friendly filtering
- Listing-details pages
- Multilingual voice-guided search

Only listings with ListingStatus.PUBLISHED appear publicly.

Tenant Portal

- Dashboard
- Leases and lease details
- Payments and payment details
- Maintenance requests
- Complaints

- Secure documents
- Messages
- Appointments
- Notifications
- Emergency-contact details
- Accessibility requirements

Landlord Portal

- Property creation and editing
- Unit management
- Tenant activity
- Maintenance and complaint management
- Lease and document management
- Listing analytics
- Maintenance expense tracking

Automation

- Rent reminders
- Lease-renewal reminders
- Appointment reminders
- Email, phone, and WhatsApp foundations
- Service-request messaging
- Routine reports
- Image-processing foundations

External communication providers must be configured before production use.

Technology

- Python 3.11+
- FastAPI
- Uvicorn
- Jinja2
- SQLAlchemy
- SQLite for local development
- Starlette SessionMiddleware
- python-multipart
- python-dotenv
- HTML, CSS, and JavaScript
- Web Speech API

Folder Structure

```
smart property ai/
├─ app.py
├─ config.py
├─ requirements.txt
├─ README.md
├─ .env
├─ database/
│   ├── __init__.py
│   ├── database.py
│   └── models.py
├─ routes/
│   ├── __init__.py
│   ├── main_routes.py
│   ├── auth_routes.py
│   ├── listing_routes.py
│   ├── application_routes.py
│   ├── ai_routes.py
│   └── tenant_routes.py
├─ services/
│   ├── __init__.py
│   ├── auth_service.py
│   ├── property_service.py
│   ├── unit_service.py
│   ├── listing_service.py
│   ├── application_service.py
│   ├── appointment_service.py
│   └── tenant_service.py
├─ security/
│   ├── __init__.py
│   ├── passwords.py
│   └── permissions.py
├─ templates/
│   ├── base.html
│   ├── index.html
│   ├── contact.html
│   ├── auth/
│   ├── legal/
│   ├── listings/
│   │   ├── listings.html
│   │   ├── listing_details.html
│   │   ├── search_results.html
│   │   ├── guided_search.html
│   │   ├── application.html
│   │   └── appointment.html
│   ├── ai/
│   │   └── assistant.html
│   ├── landlord/
│   └── tenant/
│       ├── dashboard.html
│       ├── leases.html
│       ├── lease_details.html
│       ├── payments.html
│       ├── payment_details.html
│       ├── maintenance_requests.html
│       ├── add_maintenance_request.html
│       ├── maintenance_request_details.html
│       ├── complaints.html
│       ├── add_complaint.html
│       ├── complaint_details.html
│       ├── documents.html
│       ├── document_details.html
│       ├── messages.html
│       ├── message_details.html
│       ├── appointments.html
│       ├── appointment_details.html
│       ├── notifications.html
│       └── profile.html
```

```

├── static/
│   ├── css/
│   │   ├── style.css
│   │   ├── auth.css
│   │   ├── property.css
│   │   ├── listings.css
│   │   ├── tenant.css
│   │   ├── ai-assistant.css
│   │   ├── accessibility.css
│   │   └── responsive.css
│   ├── js/
│   │   ├── main.js
│   │   ├── auth.js
│   │   ├── property.js
│   │   ├── listings.js
│   │   ├── tenant.js
│   │   ├── ai-assistant.js
│   │   ├── language-selector.js
│   │   ├── voice-forms.js
│   │   ├── speech-output.js
│   │   └── guided-search.js
│   └── images/
│       └── favicon.ico
├── uploads/
└── logs/

```

Installation on Windows

Open PowerShell in the project directory:

```
cd "C:\Users\Bolokaieimi\PycharmProjects\smart property ai"
```

Create and activate a virtual environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If activation is blocked:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

Install dependencies:

```
pip install -r requirements.txt
```

Environment Variables

Create .env in the project root:

```

ENVIRONMENT=development
DEBUG=true
APP_NAME=Smart Property AI
APP_VERSION=1.0.0
SECRET_KEY=replace-with-a-long-random-secret
SESSION_COOKIE_NAME=smart_property_session
SESSION_MAX_AGE=604800
DATABASE_URL=sqlite:///./smart_property_ai.db

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=your-email@gmail.com
SMTP_PASSWORD=your-google-app-password
SMTP_FROM_EMAIL=your-email@gmail.com
CONTACT_RECIPIENT_EMAIL=your-email@gmail.com

```

Generate a development secret:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Never commit .env or real credentials.

Database

The database files are:

```
database/database.py
database/models.py
```

Do not create library/models.py and do not run database/models.py directly.

Start the project through app.py.

Important models include:

- User
- Property
- Unit
- Listing
- ListingEvent
- TenantProfile
- Lease
- Payment
- MaintenanceRequest
- Complaint
- Document
- Message
- Appointment
- Notification
- RentalApplication
- ViewingAppointment

UUID identifiers use String(36). Route and service ID parameters must use str, not int.

Router Registration

Import and register each router exactly once in app.py:

```
from routes.main_routes import router as main_router
from routes.auth_routes import router as auth_router
from routes.listing_routes import router as listing_router
from routes.application_routes import router as application_router
from routes.ai_routes import router as ai_router
from routes.tenant_routes import router as tenant_router

app.include_router(main_router)
app.include_router(auth_router)
app.include_router(listing_router)
app.include_router(application_router)
```

```
app.include_router(ai_router)
app.include_router(tenant_router)
```

The project must contain only one `app = FastAPI(...)` definition. Defining it again discards previously registered middleware and routers.

Jinja2 Templates

`base.html` needs one main content block:

```
<main id="main-content" class="site-main" tabindex="-1">
    {% block content %}
    {% endblock %}
</main>
```

Every child template uses:

```
{% extends "base.html" %}

{% block title %}Page Title | Smart Property AI{% endblock %}

{% block content %}
<section>
    <h1>Page title</h1>
</section>
{% endblock %}
```

Separate content blocks are not required for every tenant page.

Static files use:

```
<link rel="stylesheet"
      href="{{ url_for('static', path='css/style.css') }}">

<script src="{{ url_for('static', path='js/main.js') }}" defer></script>
```

Main URLs

Public and Authentication

Method	URL	Purpose
---	---	---
GET	/	Homepage
GET	/login	Login
POST	/login	Authenticate
GET	/register	Registration
POST	/register	Create account
POST	/logout	End session
GET	/contact	Contact page
POST	/contact	Validate and deliver contact message

Listings

Method	URL	Purpose
---	---	---
GET	/apartments	Public listings

GET	/find-an-apartment	Apartment search alias
GET	/apartments/search	Search results
GET	/guided-search	Guided apartment search
GET	/apartments/{listing_id}	Listing details

Example:

/apartments/search?city=Herne&min_rent=500&max_rent=1200&bedrooms=2

AI Assistant

Method	URL	Purpose
---	---	---
GET	/ai-assistant	Assistant page
POST	/api/ai/chat	Submit a message
GET	/api/ai/status	Assistant status

Tenant Portal

Method	URL
---	---
GET	/tenant/dashboard
GET	/tenant/leases
GET	/tenant/payments
GET	/tenant/maintenance-requests
GET	/tenant/complaints
GET	/tenant/documents
GET	/tenant/messages
GET	/tenant/appointments
GET	/tenant/notifications
GET	/tenant/profile

Private tenant, landlord, application, document, and AI routes require an authenticated session.

Contact Form

The template is templates/contact.html. Its GET and POST routes belong in routes/main_routes.py and must be named:

```
contact_page
contact_submit
```

The navigation link is:

```
<a href="{{ url_for('contact_page') }}">Contact</a>
```

The contact backend provides:

- CSRF validation

- Server-side validation
- Email validation
- Category allow-list validation
- Privacy consent
- Header-injection protection
- Form-value preservation after errors
- Safe SMTP failure responses
- CSRF rotation after successful delivery

Contact-message contents, credentials, passwords, and tokens must never be written to logs.

Privacy and Security

- Signed session cookies
- Secure production cookies
- SameSite protection
- Password hashing
- CSRF protection
- Role-based authorization
- UUID identifiers
- Input validation
- Private document authorization
- Security headers
- Consent records
- Audit logging
- AI-processing disclosure
- Data minimization

Never mount /uploads publicly. Return private files only through authenticated routes after verifying authorization.

Accessibility

- Semantic HTML
- Keyboard navigation
- Visible focus indicators
- Screen-reader labels
- High-contrast support
- Reduced-motion support

- Responsive layouts
- Voice-enabled forms
- Speech output
- Multilingual controls

The interface must remain usable when browser speech features are unavailable.

Run the Application

From the project root:

```
uvicorn app:app --reload
```

Open:

```
http://127.0.0.1:8000
```

Useful development URLs:

```
http://127.0.0.1:8000/health
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/contact
http://127.0.0.1:8000/apartments
http://127.0.0.1:8000/guided-search
http://127.0.0.1:8000/ai-assistant
http://127.0.0.1:8000/tenant/dashboard
```

Testing

Startup

- Confirm Uvicorn starts without import errors.
- Confirm /health returns a healthy response.
- Confirm /docs displays all routers.
- Confirm database tables are created.
- Confirm static files return HTTP 200.

Authentication

- Register an account.
- Log in with valid credentials.
- Confirm invalid credentials are rejected.
- Test Remember Me.
- Test password reset.
- Confirm logout clears the session.

Listings

- Create a property, unit, and published listing.
- Open /apartments.
- Test all filters.
- Open listing details.

- Test guided search and voice input.

Tenant Portal

- Confirm unauthenticated access redirects to login.
- Review leases and payments.
- Submit maintenance requests and complaints.
- Review documents, messages, appointments, and notifications.
- Update emergency-contact and accessibility information.

Contact

- Open /contact.
- Submit valid information.
- Confirm email delivery.
- Test invalid email and missing consent.
- Confirm form values remain after an error.
- Confirm message contents do not appear in logs.

Troubleshooting

TemplateNotFound

For:

```
jinja2.exceptions.TemplateNotFound: 'listings/guided_search.html'
```

confirm:

```
templates/listings/guided_search.html
```

Start Uvicorn from the project root.

SessionMiddleware

```
from starlette.middleware.sessions import SessionMiddleware
```

```
app.add_middleware(
    SessionMiddleware,
    secret_key=SECRET_KEY,
)
```

Missing python-multipart

```
pip install python-multipart
```

Missing RentalApplication

Confirm database/models.py contains both:

```
class RentalApplication(Base):
    ...
```

```
class ViewingAppointment(Base):
    ...
```

RelationshipDeclared Is Not Callable

When a relationship named `property` shadows Python's decorator:

```
import builtins

@builtins.property
def applicant_full_name(self):
    ...
```

Empty Listings

Only `ListingStatus.PUBLISHED` listings appear publicly.

AI Development Roadmap

The current assistant is a local rule-based foundation. Planned custom AI work:

1. Define supported property-management tasks.
2. Create consent-aware, anonymized training data.
3. Remove direct identifiers and sensitive tenant data.
4. Create multilingual training and evaluation datasets.
5. Implement training and evaluation scripts.
6. Test privacy, safety, bias, accessibility, and accuracy.
7. Require human review for important tenancy decisions.
8. Register and version approved models.
9. Connect the approved model to the assistant route.
10. Monitor quality without training on private conversations by default.

A future AI folder may contain:

```
ai/
├─ __init__.py
├─ train.py
├─ evaluate.py
├─ inference.py
├─ prompts.py
├─ safety.py
├─ datasets/
├─ checkpoints/
└─ tests/
```

`train.py` becomes necessary when custom model training begins. It is not required for the current rule-based assistant.

Production Checklist

- Set `ENVIRONMENT=production`.
- Set `DEBUG=false`.
- Configure a permanent `SECRET_KEY`.
- Use `HTTPS`.
- Configure secure `SMTP` credentials.
- Add database migrations.
- Use a managed production database when appropriate.

- Restrict CORS.
- Validate and scan uploads.
- Apply upload size limits.
- Add rate limiting and account lockout.
- Configure backups and monitoring.
- Complete privacy, legal, accessibility, and AI safety reviews.
- Never expose /uploads publicly.

Git Ignore

Add to .gitignore:

```
.env
*.db
*.sqlite
*.sqlite3
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/
.venv/
venv/
env/
.idea/
.vscode/
logs/
uploads/
ai/checkpoints/
```

Legal Notice

Smart Property AI assists with property-management workflows. It does not replace professional legal, financial, housing, safety, or emergency advice. Important decisions require authorized human review.

License

Project owner and author: Ebi Emmerich-Adehor.

Copyright (c) 2026 Ebi Emmerich-Adehor. All rights reserved.

No open-source license has been granted yet. Add a separate LICENSE file only after choosing the terms under which others may use or distribute this project.